

Unpaid stints and find-a-problem moves

Week 8, Lecture 2 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- When an unpaid trial is worth it and when it is exploitative
- The cold-email structure that gets a yes to a two-week free sprint
- How to find a real problem at a target company
- Building the fix without asking permission
- Build-in-public as a visibility compressor
- Earning the introduction from an artifact

By the end of this lecture you can decide whether an unpaid move is right for your situation and write the email or build the artifact that creates the conversation.

The unpaid stint decision

When unpaid work is worth it

An unpaid two-week trial can be rational if all four conditions are true:

1. The company is a small startup where the founders control hiring
2. You genuinely want to work there and have no other path in
3. You can financially sustain two weeks without income
4. There is a clear and explicit conversion path: a full-time offer if the trial goes well

"Startups take off because the founders make them take off." Graham (2013) applies to applicants too. Doing something unscalable to get your first foot in is exactly this playbook.

When unpaid work is exploitative

The same move is a bad deal when:

- The company is large enough to have an HR department and a budget (they should pay you)
- There is no conversion path stated in advance (you are providing free labor with no upside)
- You cannot afford to work without income (the opportunity cost is too high relative to the probability of conversion)
- The "trial" extends indefinitely with reasons why a full offer keeps getting deferred

The test: would a lawyer look at this arrangement and call it an unpaid internship under a different name? If yes, walk away.

The decision in one frame

- Tiny startup, founders are accessible
 - Role does not exist yet but could
 - You have savings or another income
 - Explicit offer-or-out date agreed in writing
 - You would do it as a project anyway
-
- Company has 50+ employees and a budget
 - No stated conversion criteria
 - You need the income to survive
 - The arrangement keeps extending
 - You are doing core work, not a project

Unpaid stint makes sense

Unpaid stint is a trap

Decide before you send the email. If you cannot put the conversion criteria in writing, the answer is no.

Cold-emailing to work free for two
weeks

The email that gets a yes

Under 150 words. Every word earns its place.

Subject: Two weeks free to fix [specific problem]

Hi [Name],

I use [Product] every day and noticed [specific friction or gap]. I am a CS senior at [University] with background in [relevant skill]. I would like to spend two weeks building a fix for this, free, so you can see what I can do before committing to anything.

I am not asking for a job. I am offering to solve a real problem. If the work is useful after two weeks, I would love to talk about what comes next.

Can we get on a 15-minute call this week to see if this makes sense?

[Name] | [GitHub] | [portfolio]

What makes this email work

Five structural choices that get replies:

1. **Subject line names the problem.** The recipient knows immediately whether this is relevant to them.
2. **First sentence proves you use the product.** Not "I have always been interested in your company." You live with the friction.
3. **The offer is explicit.** Two weeks, free, a specific deliverable. Not "I would love to contribute."
4. **Low commitment ask.** A 15-minute call is easy to say yes to. A job application is not.
5. **No resume attachment.** The email IS the first signal. The GitHub link is the second.

Graham (2013): "The most successful founders I've talked to don't wait for permission."

Find a problem and solve it

Where to look for real problems

Four surfaces where real product friction lives:

1. **Use the product yourself.** Note every point where you curse at the screen or work around a limitation.
2. **Read the issue tracker.** Public repos show what users have complained about for months without a fix.
3. **Read the engineering blog.** Posts about "how we solved X" often signal that Y is the next unsolved problem.
4. **Read the support forum or Reddit.** Real users complain in plain language about real pain.

The goal is a problem specific enough that your fix is unambiguous. "Improve the onboarding experience" is too broad. "The signup form loses form state on validation error" is a problem you can solve this weekend.

Building without permission

Graham (2023): "If you're excited about some possibility that everyone else ignores, and you have enough expertise to say precisely what they're all overlooking, that's as good a bet as you'll find."

The build-without-permission sequence:

1. Identify a problem specific enough to have a two-day fix
2. Build the fix: a patch, a prototype, a script, or a detailed proposal with a working proof of concept
3. Document the problem and the fix clearly (a GitHub repo with a README, or a blog post)
4. Send the artifact to a specific human at the company, not a contact form

The artifact is the application. The email is the context.

What a good artifact looks like

- "I built a demo app using your API"
 - A Todo app in your stack of choice
 - A slide deck about what you would do if hired
 - A redesign mockup with no working code
-
- A patch that fixes issue #847 in their public repo
 - A script that improves a documented workflow gap
 - A write-up that diagnoses an architectural tradeoff they have publicly wrestled with

Weak artifact

Strong artifact

The strong artifact proves you understand the codebase or product at a level most applicants do not reach.

Build-in-public as attention leverage

Why shipping publicly compounds your visibility

Levels (2019): "Build your startup in public... you get so much attention and press, and definitely try this."

Applied to job search: the artifact does not need to stay private. A public write-up, demo, or tool works in two directions:

1. **Inbound attention.** Engineers at your target company may find it before you send it to them.
2. **Third-party credibility.** A public GitHub repo or blog post that others engage with arrives with social proof attached.

The same artifact you send in a cold email can also be the artifact that earns you an introduction from someone who found it first.

The build-in-public sequence for job search

1. Identify the problem (use the product, read the tracker)
2. Build the fix or write-up (two days to one week)
3. Publish publicly: GitHub repo, blog post, or X thread
4. Tag the company's engineering account if appropriate
5. Send the artifact link directly to a real person at the company
6. Respond to any engagement within 24 hours

The public step is optional but high-leverage. If you are not comfortable posting publicly, send the artifact privately first and post later if the conversation stalls.

Earning the introduction

From artifact to conversation to referral

The artifact starts the conversation. The conversation earns the introduction. The introduction is worth more than the job application.

Three paths from artifact to introduction:

1. The person you emailed replies, connects you to the hiring manager, and mentions the artifact. This is a warm introduction.
2. Someone else finds the artifact publicly and mentions your name in a relevant Slack channel or team meeting. This is an inbound introduction.
3. A maintainer whose project you contributed to works at the target company and mentions your PR history when a relevant role opens. This is a reputation introduction.

All three require the artifact to exist first.

Take-aways

1. An unpaid stint is worth it only when the company is small, you can afford it, and a conversion path is explicit in writing. Otherwise it is free labor with no return.
2. A cold email works when it names a specific problem, offers a specific deliverable, and asks for a 15-minute call. Not a job application: an offer to solve a problem.
3. Find a problem by using the product, reading the issue tracker, and reading the engineering blog. Build the minimum artifact that proves you can solve it.
4. Shipping publicly adds an inbound dimension to the same artifact. The same work earns attention from two directions at once.
5. The artifact is not the offer. It is the credential that earns the conversation, and the conversation earns the introduction.

"Actually startups take off because the founders make them take off. There may be a handful that just grew by themselves, but usually it takes some sort of push to get them going."

Paul Graham, *Do Things that Don't Scale* (2013)

What's next

- **Section (this week):** Triage and ship. You have already decided which move to make. Bring the artifact, PR, or sent emails to peer review.
- **Reading:** Week 8 reading at </c/landing-the-offer-26au/readings/wk08>
- **Next week (Week 9):** Running the formal pipeline: recruiter conversations, application cadence, and negotiation
- **HW 4 (system design write-up):** due this week